IPSSI

2e année Bachelor Informatique

Mathématiques appliquées à l'Intelligence Artificielle

Fiche d'exercices

Jour 2 — Optimisation

Auteur : Mohamed EL AFRIT

www.mohamedelafrit.com

Année académique : 2025–2026

© 2025 Mohamed EL AFRIT — IPSSI

Ce contenu est distribué sous licence

Creative Commons BY-NC-SA 4.0

https://creativecommons.org/licenses/by-nc-sa/4.0/deed.fr

Rappels — Formules du Jour 2

Fonction de coût : $J(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$ **Mise à jour :** $w \leftarrow w - \alpha \cdot \frac{\partial J}{\partial w}$

Dérivées utiles : $(c)' = 0$, $(ax+b)' = a$, $(x^2)' = 2x$, $(ax^2)' = 2ax$, $((x-a)^2)' = 2(x-a)$

Gradients régression ($\hat{y}_i = w_1 x_i + w_0$) : $\frac{\partial J}{\partial w_1} = \frac{-2}{n}\sum x_i(y_i - \hat{y}_i)$ $\frac{\partial J}{\partial w_0} = \frac{-2}{n}\sum(y_i - \hat{y}_i)$

Dataset salaire (30 dev, régression expérience → salaire) : $\hat{y} = 2{,}07x + 31{,}27$, MSE $= 20{,}46$.

Exercices crayon-papier — Niveau fondamental

Exercice 1 — ★☆☆ **Fondamental** — Calculer le MSE

Un modèle a produit les prédictions suivantes :

i	Salaire réel y_i (k€)	Prédiction $\hat{y}_i$ (k€)
1	30	33
2	42	40
3	55	52
4	38	44

a) Calculez l'erreur $e_i = y_i - \hat{y}_i$ pour chaque observation.

b) Calculez l'erreur au carré e_i^2 pour chaque observation.

c) Calculez le MSE : $J = \frac{1}{n}\sum e_i^2$.

d) L'observation $i = 4$ a la plus grande erreur. Si on corrigeait cette seule prédiction pour qu'elle soit parfaite ($\hat{y}_4 = 38$), quel serait le nouveau MSE ? De combien a-t-il diminué ?

Exercice 2 — ★☆☆ **Fondamental** — Calculer des dérivées

Calculez la dérivée $f'(x)$ des fonctions suivantes. Précisez la règle utilisée à chaque fois.

a) $f(x) = 3x^2$

b) $f(x) = x^2 - 6x + 9$

c) $f(x) = 2x + 5$

d) $f(x) = (x-7)^2$

e) $f(x) = 5(x-2)^2 + 3$

Pour les fonctions a), b) et d), calculez la valeur de x où $f'(x) = 0$. Que représente ce point ?

Exercice 3 — ★☆☆ **Fondamental** — Une itération de descente de gradient

On considère la fonction de coût $J(w) = (w-4)^2$.

a) Calculez la dérivée $J'(w)$.

b) On part de $w_0 = 0$ avec un taux d'apprentissage $\alpha = 0{,}2$. Calculez la dérivée en w_0 : $J'(0) = ?$

c) Appliquez la règle de mise à jour : $w_1 = w_0 - \alpha \cdot J'(w_0)$. Quelle est la nouvelle valeur w_1 ?

d) Calculez $J(w_0)$ et $J(w_1)$. Le coût a-t-il diminué ?

e) Le minimum théorique est $w^* = 4$. À quel pourcentage du chemin êtes-vous après une seule itération ?

Exercice 4 — ★★★ **Fondamental** — Trois itérations complètes

On considère $J(w) = w^2$ (minimum en $w^* = 0$). Taux d'apprentissage $\alpha = 0{,}1$.

a) Calculez $J'(w)$.

b) En partant de $w_0 = 5$, remplissez le tableau suivant :

Itération	w	$J'(w)$	$w_{\text{new}} = w - \alpha J'(w)$	$J(w)$
0	5,00	?	?	?
1	?	?	?	?
2	?	?	?	?

c) Après 3 itérations, de combien w s'est-il rapproché de $w^* = 0$? Calculez le ratio w_3/w_0.

d) **Bonus :** montrez que $w_k = w_0 \cdot (1 - 2\alpha)^k$ pour cette fonction. Vérifiez avec vos résultats.

Exercices crayon-papier — Niveau intermédiaire

Exercice 5 — ★★★ **Intermédiaire** — Dérivées partielles de la régression

On considère 3 observations avec le modèle $\hat{y}_i = w_1 x_i + w_0$:

i	x_i (expérience)	y_i (salaire, k€)
1	2	32
2	5	44
3	10	55

a) Avec $w_1 = 3$ et $w_0 = 25$, calculez les prédictions $\hat{y}_i$ et les erreurs $e_i = y_i - \hat{y}_i$.

b) Calculez la dérivée partielle par rapport à w_1 :

$$\frac{\partial J}{\partial w_1} = \frac{-2}{n} \sum_{i=1}^{3} x_i \cdot e_i$$

c) Calculez la dérivée partielle par rapport à w_0 :

$$\frac{\partial J}{\partial w_0} = \frac{-2}{n} \sum_{i=1}^{3} e_i$$

d) Avec $\alpha = 0{,}01$, calculez les nouveaux poids : $w_1' = w_1 - \alpha \frac{\partial J}{\partial w_1}$ et $w_0' = w_0 - \alpha \frac{\partial J}{\partial w_0}$.

e) Dans quelle direction chaque poids a-t-il bougé ? Est-ce cohérent ?

Exercice 6 — ★★★ **Intermédiaire** — Diagnostiquer un problème de convergence

Un étudiant a exécuté une descente de gradient sur $J(w) = (w-3)^2$ avec $\alpha = 1{,}5$ et obtient les résultats suivants :

Itération	w	$J(w)$
0	0,00	9,00
1	9,00	36,00
2	−15,00	324,00
3	51,00	2 304
4	−93,00	9 216

a) Vérifiez le calcul de l'itération $0 \to 1$: $w_1 = w_0 - \alpha \cdot J'(w_0)$.
b) Le coût J augmente-t-il ou diminue-t-il ? Quel est le problème ?
c) Avec quelle valeur de α la convergence serait-elle assurée pour cette fonction ? *Indice :* pour $J(w) = (w-a)^2$, la convergence nécessite $0 < \alpha < 1$.
d) Recalculez les 3 premières itérations avec $\alpha = 0{,}1$. Le coût diminue-t-il cette fois ?

Exercice 7 — ★★☆ **Intermédiaire** — Comparer deux taux d'apprentissage

On minimise $J(w) = (w-5)^2$ en partant de $w_0 = 0$. Deux étudiants utilisent des α différents.

a) **Étudiant A** ($\alpha = 0{,}05$) : calculez w_1, w_2, w_3 et les coûts associés.
b) **Étudiant B** ($\alpha = 0{,}3$) : calculez w_1, w_2, w_3 et les coûts associés.
c) Remplissez un tableau comparatif :

	w_1	w_2	w_3	$J(w_3)$
$\alpha = 0{,}05$	?	?	?	?
$\alpha = 0{,}3$	?	?	?	?

d) Quel étudiant se rapproche le plus vite de $w^* = 5$? Lequel risque-t-il de dépasser le minimum avec un α plus grand ?

Exercices Python guidés — Niveau intermédiaire

Exercice 8 — ★★☆ **Intermédiaire** — Descente de gradient 1D from scratch [Python guidé]

Complétez le code pour minimiser $J(w) = (w-3)^2$ par descente de gradient :

» Squelette -- Descente de gradient 1D

```
import numpy as np

# Fonction de cout et sa derivee
def J(w):
    return ______   # A COMPLETER : (w - 3) ** 2

```

```python
def dJ(w):
    return ______   # A COMPLETER : derivee de J

# Parametres
w = 0.0                # point de depart
alpha = ______         # A COMPLETER : taux d'apprentissage (essayer 0.1)
n_iter = 20            # nombre d'iterations

# Boucle de descente de gradient
for i in range(n_iter):
    grad = ______          # A COMPLETER : calculer la derivee
    w = ______             # A COMPLETER : mise a jour de w
    if i < 5 or i == n_iter - 1:
        print(f"Iter {i+1:2d} : w = {w:.4f}, J = {J(w):.4f}")

print(f"\nResultat : w = {w:.4f} (optimal : 3.0)")
print(f"Erreur : |w - 3| = {abs(w - 3):.6f}")
```

Exercice 9 — ★★☆ **Intermédiaire** — Visualiser la convergence [Python guidé]

Complétez le code pour tracer la courbe de coût à chaque itération :

» Squelette -- Visualisation de la convergence

```python
import numpy as np
import matplotlib.pyplot as plt

def J(w): return (w - 3) ** 2
def dJ(w): return 2 * (w - 3)

w = -2.0
alpha = 0.15
n_iter = 25

# Sauvegarder l'historique
hist_w = [w]
hist_J = [J(w)]

for i in range(n_iter):
    w = w - alpha * dJ(w)
    hist_w.append(______)    # A COMPLETER
    hist_J.append(______)    # A COMPLETER

# --- Graphique 1 : trajectoire sur la courbe ---
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))

w_range = np.linspace(-3, 8, 200)
ax1.plot(w_range, J(w_range), 'b-', lw=2)
ax1.plot(hist_w, hist_J, 'ro-', markersize=5,
         label='Iterations')
```

```
ax1.set_xlabel('w'); ax1.set_ylabel('J(w)')
ax1.set_title('Trajectoire sur la courbe')
ax1.legend(); ax1.grid(True, alpha=0.3)

# --- Graphique 2 : J en fonction du numero d'iteration ---
ax2.plot(range(len(hist_J)), ______, 'g-o',  # A COMPLETER
         markersize=4)
ax2.set_xlabel('______')      # A COMPLETER
ax2.set_ylabel('______')      # A COMPLETER
ax2.set_title('Courbe de convergence')
ax2.grid(True, alpha=0.3)

plt.tight_layout(); plt.show()
```

Exercice 10 — ★★☆ **Intermédiaire** — Gradient descent sur le dataset salaire [Python guidé]

Complétez l'implémentation de la descente de gradient pour la régression linéaire (expérience → salaire) :

» Squelette -- Régression par descente de gradient

```
import numpy as np
import matplotlib.pyplot as plt

# Dataset
x = np.array([0,1,1,2,2,0,3,1,4,5,5,6,6,7,7,4,
    8,5,9,10,10,11,12,9,13,11,15,17,20,18], dtype=float)
y = np.array([28.0,26.5,35.0,32.0,38.0,25.0,34.5,42.0,
    38.5,44.0,41.0,48.0,43.5,50.0,39.0,47.0,51.0,40.0,
    52.0,55.0,48.5,58.0,60.0,42.0,62.0,54.0,65.0,68.0,
    72.0,58.0])
n = len(x)

# Parametres initiaux
w1, w0 = 0.0, 0.0
alpha = 0.001
n_iter = 5000
hist_mse = []

for it in range(n_iter):
    # 1. Predictions
    y_pred = ______                          # A COMPLETER

    # 2. Erreurs
    errors = ______                          # A COMPLETER

    # 3. Gradients
    dw1 = -2/n * np.sum(______)              # A COMPLETER
    dw0 = -2/n * np.sum(______)              # A COMPLETER

    # 4. Mise a jour
```

 CC BY-NC-SA 4.0

```python
    w1 = w1 - ______                        # A COMPLETER
    w0 = w0 - ______                        # A COMPLETER

    hist_mse.append(np.mean(errors ** 2))

print(f"Resultat : y = {w1:.4f} * x + {w0:.4f}")
print(f"Attendu  : y = 2.0672 * x + 31.2830")
print(f"MSE final = {hist_mse[-1]:.2f}")

# Graphique
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))
ax1.plot(hist_mse); ax1.set_xlabel('Iteration')
ax1.set_ylabel('MSE'); ax1.set_title('Convergence')
ax1.grid(True, alpha=0.3)

ax2.scatter(x, y, c='blue', s=30)
xl = np.linspace(0, 21, 100)
ax2.plot(xl, ______, 'r-', lw=2,      # A COMPLETER
         label=f'GD: y={w1:.2f}x+{w0:.2f}')
ax2.set_xlabel('Experience'); ax2.set_ylabel('Salaire')
ax2.set_title('Regression'); ax2.legend()
ax2.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()
```

Exercices Python autonomes — Niveau avancé

Exercice 11 — ★★★ **Avancé** — Comparer 4 taux d'apprentissage [Python autonome]

Écrivez un programme Python complet qui :

a) Définit la fonction $J(w) = (w-5)^2$ et sa dérivée.

b) Exécute la descente de gradient pour 4 valeurs de α : 0,001, 0,01, 0,1 et 0,9. Point de départ : $w_0 = -3$, nombre d'itérations : 50.

c) Trace **deux graphiques** côte à côte :
   — À gauche : les 4 trajectoires sur la courbe $J(w)$ (une couleur par α)
   — À droite : les 4 courbes de convergence (J vs numéro d'itération)

d) Affiche un tableau récapitulatif : pour chaque α, la valeur finale de w et le MSE final.

e) Répondez en commentaire : quel α est le meilleur ? Pourquoi ? Y a-t-il un α qui diverge ?

⚠ **Consignes**

Utilisez uniquement NumPy et Matplotlib. Commentez en français. Utilisez une boucle sur les valeurs de α pour éviter la répétition de code.

Exercice 12 — ★★★ **Avancé** — Gradient descent vs scikit-learn [Python autonome]

Écrivez un programme qui compare deux méthodes pour trouver la droite de régression (expérience → salaire) sur le dataset fil rouge complet :

a) **Méthode 1 — From scratch :** implémentez la descente de gradient complète (départ $w_1 = w_0 =$

0, $\alpha = 0{,}001$, 10 000 itérations). Enregistrez w_1, w_0 et le MSE à chaque itération.

b) **Méthode 2 — scikit-learn :** utilisez `LinearRegression` de scikit-learn pour trouver les poids optimaux. *Rappel :* `X` doit être de shape `(n, 1)` pour scikit-learn.

c) Affichez un tableau comparatif :

Méthode	w_1	w_0	MSE
Gradient descent (10k)	?	?	?
scikit-learn	?	?	?

d) Tracez un graphique avec :
- le nuage de points du dataset,
- la droite from scratch (rouge),
- la droite scikit-learn (verte),
- une légende avec le MSE de chaque méthode.

e) Tracez la courbe de convergence du MSE (itérations vs MSE) avec une ligne horizontale indiquant le MSE de scikit-learn.

f) Répondez en commentaire : les deux méthodes trouvent-elles le même résultat ? Combien d'itérations faut-il approximativement pour que la descente de gradient atteigne 99% de la performance de scikit-learn ?

Indice

Pour scikit-learn : `from sklearn.linear_model import LinearRegression`. L'entrée `X` doit avoir le shape `(30, 1)` : utilisez `x.reshape(-1, 1)`.

Répartition : 4 exercices ★ fondamentaux 3 exercices ★★ intermédiaires 3 guidés Python ★★ 2 autonomes ★★★

© 2025 Mohamed EL AFRIT — IPSSI • www.mohamedelafrit.com • CC BY-NC-SA 4.0